

GRControl — Gonioreflectometer User Manual

Complete operating guide for the **GRControl** software that drives the tabletop gonioreflectometer. It covers every screen, button, and field in the web interface, the recommended operating workflows, the files the instrument writes, and troubleshooting.

Who this is for: operators running measurements. No programming is required to use the instrument. The final sections (Files, Troubleshooting, Reference) are useful for anyone analyzing the captured data.

Table of contents

1. [What the instrument does](#)
2. [Hardware at a glance](#)
3. [Starting the software](#)
4. [The interface — overview](#)
5. [Top bar: status & emergency stop](#)
6. [Sidebar: Connection, Home, Log](#)
7. [Encoder Readings](#)
8. [Camera Control](#)
9. [Manual Motor Control](#)
10. [LED Control](#)
11. [Scan Configuration \(grid scan\)](#)
12. [New Scan Configuration \(single geometry\)](#)
13. [Scan Progress & Captured Images](#)
14. [Output files & folder layout](#)
15. [Recommended operating procedure](#)
16. [Diagnostics: encoder jitter](#)

17. [Troubleshooting](#)
 18. [Safety](#)
 19. [Quick reference](#)
-

1. What the instrument does

A **gonioreflectometer** photographs a sample under controlled **illumination angle** and **viewing (camera) angle**. GRControl coordinates three things for each measurement:

- **Camera arm** (Motor 1) — rotates the camera to a viewing angle.
- **LED arc** (Motor 2) — rotates an arc carrying **7 LEDs** to an illumination angle. The LEDs are fixed on the arc at **0°, 10°, 20°, 30°, 40°, 50°, 60°** (LED 1 → LED 7), so lighting a different LED changes the incidence angle by a known offset relative to the arc's position.
- **Camera** — an IDS industrial camera that captures the sample and saves each frame with a **JSON sidecar** recording exactly where every axis was.

The software runs as a **local web application**: a Python backend talks to the hardware, and you operate everything from a browser. Nothing is installed in the browser — just open the page.

2. Hardware at a glance

Part	Role	Notes
ESP32 microcontroller	Drives both stepper motors, the 7 LEDs, and reads the encoders	Connects to the PC by USB serial or WiFi/TCP

Motor 1	Camera arm	Has a motor-shaft encoder (AS5600) and a high-precision output encoder (OME85)
Motor 2	LED arc	Has a motor-shaft encoder (AS5600) and an output-arc encoder (AS5600 #3)
7 LEDs	Illumination	Each independently switchable, brightness 0–255 (PWM)
IDS camera	Image capture	Colour sensor, 12-bit; connects by USB 3

Motor numbering — memorize this: Motor 1 = Camera, Motor 2 = LED arc. This is the firmware-confirmed mapping used everywhere in the UI and the saved files.

Angles are in degrees. "Home" is the zero reference you set manually (see [§6](#)). The software displays and records angles as signed values around home (e.g. -90° , $+45^\circ$), so a homed axis reads a stable 0.000° rather than flickering across a 0/360 seam.

3. Starting the software

The backend and the hardware normally run on the **rig PC** (the laptop attached to the instrument and camera). On that PC:

```
# one-time setup
python -m venv .venv
source .venv/bin/activate           # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# every session
python run.py
```

Then open <http://localhost:8000> in a browser on that PC. If you operate from another computer on the same network, use the rig PC's IP address instead of `localhost`.

The camera and motors are only reachable from the PC physically wired to them. Running the browser elsewhere still shows the UI, but the hardware must be attached to the machine running `python run.py`.

The camera SDK (IDS Peak) is optional — the backend starts and all motor/LED functions work even if no camera is attached; the camera simply stays "off".

4. The interface — overview

The page has three regions:

- **Top bar** (across the top) — live connection/camera/scan status and the **E-STOP** button.
- **Sidebar** (left) — Connection, Reference/Home, and the event Log. Each section header collapses when clicked.
- **Main area** (center, scrolls) — the working cards, in order:
 1. Encoder Readings
 2. Camera Control
 3. Scan Configuration (grid scan)
 4. New Scan Configuration (single geometry)
 5. Scan Progress (appears only while scanning)
 6. Captured Images (gallery)
 7. Manual Motor Control
 8. LED Control

A **theme toggle** (☾ / ☀) in the top bar switches light/dark and remembers your choice.

5. Top bar: status & emergency stop

Element	Meaning
Connection pill	Red = disconnected from the ESP32. Green = connected (shows <code>serial</code> or <code>tcp</code>).
Camera pill	Red = camera off. Green = camera open (shows the model).
Scanning pill	Appears (amber) only while a scan runs; shows position progress.
 / 	Toggle dark/light theme.
 E-STOP	Emergency stop. Immediately stops all motors and turns all LEDs off .

E-STOP is always available. It does not abort a scan's bookkeeping, but it halts motion and light instantly — use it whenever anything looks wrong, then Abort the scan if one is running.

6. Sidebar: Connection, Home, Log

Connection

Two tabs:

- **Serial** — pick a port from the dropdown (click ↻ **Refresh ports** to populate it), then **Connect**. This is the usual USB connection.
- **WiFi / TCP** — type the ESP32's **IP address**, then **Connect**. Use this when the ESP32 is on the network instead of USB.

After connecting, the button becomes **Reconnect** and a **Disconnect** button appears. The connection pill turns green.

If **Refresh ports** shows nothing, the ESP32 isn't detected — check the USB cable/driver and that no other program holds the port.

Reference / Home

Sets the **zero reference** for the axes. **Manually position the instrument to your desired zero**, then:

- **Set Home (Both)** — zeroes both axes at their current positions.
- **Home LED / Home Camera** — zero one axis only.

Do this at the start of a session, after any manual repositioning, or whenever the encoder readings no longer match physical zero.

Log

A running event log (connect/disconnect, moves, captures, errors). Colour-coded: normal, ✓ success, △ warning, ✕ error. **Clear** empties it. This is the first place to look when something doesn't behave as expected.

7. Encoder Readings

Live dashboard, updated ~10× per second, split into **Camera** (left) and **LED** (right) columns:

Reading	Source	Meaning
Motor 1 — Camera	AS5600 #1	Camera motor-shaft angle
Camera Final	OME85	Camera output-arm angle (the true viewing angle)
Camera Error	—	Output minus motor-shaft, normalized to (−180°, 180°]
Motor 2 — LED	AS5600 #2	LED motor-shaft angle

LED Arc Final	AS5600 #3	LED arc angle from home (true illumination angle)
LED Arc Error	—	Output minus motor-shaft, normalized

The **error** value is colour-coded: green ($< 0.5^\circ$), amber ($< 2^\circ$), red ($\geq 2^\circ$). A **—** means that encoder isn't reporting (not wired/available) — motion still works open-loop, but precise landing can't be verified.

Measure encoder jitter (bottom of the card) runs a diagnostic — see [§16](#).

8. Camera Control

Left = controls, right = **Live Preview**.

Opening and settings

- **Open Camera** — connects to the IDS camera. On success the camera pill turns green and the exposure/gain shown in the fields are applied immediately, so the camera starts at a known state.
- **Close Camera** — releases the camera.
- **Exposure (μs)** — integration time in microseconds. Higher = brighter but slower and more motion-sensitive.
- **Gain** — analog gain (0–26). Raises brightness but also noise; prefer exposure over gain for image quality.
- **Apply Settings** — pushes the current exposure & gain to the camera.
- **Mirror X / Flip Y** — tick and click **Apply** to flip the image horizontally/vertically in hardware.

Changing settings pauses the live preview for a moment. Writing camera parameters mid-stream can stall this camera's USB link, so the software quiets the stream during the write. A brief preview freeze on Apply is normal.

Image Format & Output Folder

- **Image Format** — TIFF, PNG, JPEG, or BMP. **TIFF is the measurement format** (full 12-bit raw data). See [§14](#).
- **Output Folder** — where **Save Image** writes (default `. / captures`, relative to where the backend runs on the rig PC).

Live Preview

- ► **Live** — start the continuous MJPEG preview (toggles to **II Stop**). Only one live preview at a time.
- ↺ **Refresh** — grab a single snapshot without streaming.
- 📁 **Save** — capture one frame **to disk** right now, in the selected format, with a JSON sidecar of the current encoder readings. The camera-mounting-slit angle from the Scan Configuration card is recorded too. A thumbnail lands in the **Captured Images** gallery.

The preview is a downscaled, colour-corrected JPEG **for framing only**. The saved TIFF is the real, full-depth raw data — the preview's brightness/colour is not what gets measured.

Save is blocked while a scan is running (the scan owns the camera). Wait for the scan to finish or abort it first.

9. Manual Motor Control

Two identical panels — **Motor 1 (Camera)** and **Motor 2 (LED Arc)**. A ● **MOVING** badge shows while an axis is in motion.

Jog (relative nudges)

- **Jog — degrees** — how far each jog step moves (output-shaft degrees).
- **Jog speed (%)** — jog speed.
- ◀ **CCW** / **CW** ▶ — jog that direction by the set amount.

- **Reverse direction** — flips that motor's rotation sense (if a motor turns the "wrong" way relative to your convention). This setting is remembered on the ESP32 and reflected back in the checkbox.

Move to absolute angle

- **Move to absolute angle + Go** — drive the axis to a specific angle measured from home.
- **Precise landing (encoder closed-loop)** — when ticked, after the coarse move the software nudges the motor until the **output encoder** reads the target within tolerance, then reports where it actually landed (achieved angle, residual error, iterations). Leave this on for accurate positioning; the log states whether it converged.
- **Move speed (%)** — speed for the move.
- **■ Stop Motor** — stop that axis immediately.

Precise landing needs a working output encoder. If that encoder reads , the move runs open-loop and the log warns that it couldn't verify the landing.

10. LED Control

A grid of 7 LEDs (**L1–L7**). For each:

- The **L#** button toggles that LED on/off (highlighted when on).
- The **number field** below sets its brightness (PWM **0–255**).
Changing it while the LED is on updates the brightness live.

Below the grid:

- **All On / All Off** — switch every LED together (All On uses each LED's current brightness).

LED state shown here reflects the actual hardware (driven by telemetry), so the buttons stay in sync if a scan or another action changes the

LEDs.

Recall the geometry: **L1 sits at 0° on the arc, each subsequent LED +10°, up to L7 at 60°**. Which LED you light, combined with the arc's angle, determines the illumination angle at the sample.

11. Scan Configuration (grid scan)

Automates a **grid** of measurements: every LED-arc angle × every camera angle, capturing the enabled LEDs at each usable position.

LED Arc Axis & Camera Axis

Each axis has:

- **Start (°), Stop (°), Step (°)** — define the swept positions, inclusive of Stop. If **Stop < Start** the axis sweeps the other way (e.g. 0 → -180). Step is a magnitude.
- **Speed (%)** — move speed for that axis.

LEDs Active During Scan

Seven cells, each with an **enable checkbox** and a **per-LED PWM** value, plus that LED's fixed arc angle for reference. Only the checked LEDs are lit and captured at each position, each at its set PWM. Default: only **L1** enabled.

Options

- **Camera mounting slit (°)** — the physical slit (20°–55°, 5° steps) the camera is bolted into. This is the **viewing elevation** and is **constant for the run** — it is *not* motor-driven. It's recorded in every image's sidecar (scan **and** manual Save). Set it to match how the camera is actually mounted.

- **Move both motors simultaneously** — when on, both axes move to each position together; when off, they move one after the other.
- **Settle before capture (s)** — dwell after moving (and after lighting each LED) before the camera captures, letting vibration die down and exposure stabilize. Default **5 s**.
- **Precise positioning (encoder closed-loop)** — closed-loop landing at each position (same idea as manual moves).
- **LED-arc block angle (°)** — the **occlusion guard**. The LED arc blocks the lens while it *leads* the camera within a window: a position is skipped when $0 < \text{normalize}(\theta_{\text{LED}} - \theta_{\text{camera}}) \leq \text{this angle}$. Measured at **~50°** on the rig. Set **0** to disable the guard. Skipped positions are listed in the UI and not captured.

Counts and skipped positions

- **Estimated positions / Total captures** update live as you edit the grid, LEDs, and block angle (blocked positions are subtracted).
- A **skipped-positions** disclosure lists every position the guard will drop (LED°/CAM°), so you can confirm the geometry before starting.

Running it

- **► Start Scan** — begins the run. If *some* positions are blocked, it starts and reports how many were skipped. If **every** position is blocked, it refuses with a message telling you to adjust the angles.
- **II Pause / ► Resume / ■ Abort** — appear while running (shared with the New Scan panel). Pause stops after the current step; Abort ends the run.

12. New Scan Configuration (single geometry)

A simpler workflow for measuring at **one geometry**, with the LED arc positioned **relative to the camera**, capturing **multiple photos per LED**.

- **Camera arc angle** (° — **motor target**) — absolute camera position.
- **LED arc angle relative to camera** (°) — the arc is driven to **camera + this**. Example: camera 0°, relative 45° → the LED arc goes to 45°.
- **Photos per LED** — how many frames to capture for each enabled LED (default 5). The LED stays lit across all its repeats, then turns off before the next.
- **Move speed (%)**, **LEDs Active** (enable + PWM, same as the grid scan), **Camera mounting slit**, **Image format**, **Settle before capture (s)**, **Precise positioning**, **Output folder** — same meanings as above.
- **Total captures** = enabled LEDs × photos per LED.

No occlusion guard here — you manage arc/lens blocking yourself. Check the live preview before starting.

- ► **Start New Scan** — runs it. Pause/Resume/Abort and progress appear in the shared cards below. **Only one scan (either panel) runs at a time.**

This panel is ideal for **repeatability** and **noise** studies at a fixed geometry (multiple identical frames per LED).

13. Scan Progress & Captured Images

Scan Progress (visible only while scanning)

- **Progress bar + N / total (%)** and **images captured**.
- **LED Arc / Camera** — the current commanded positions.
- **Errors** — the most recent scan errors, if any.

When the scan finishes, the panel hides and the log reports the total

images captured.

Captured Images

A live gallery of thumbnails for every frame captured (scan **and** manual Save), newest first. Each shows its filename; click one to open a **lightbox** (large view). **Clear** empties the gallery (this only clears the on-screen thumbnails — the saved files on disk are untouched).

14. Output files & folder layout

Everything is written into the **Output Folder** you set (default `./captures`, relative to the backend's working directory on the rig PC).

Image files

- **Scan captures** are named:

```
LED_<ledDeg>deg_CAM_<camDeg>deg_LED<NN>[_repMM]_<YYYY>
```

- `<ledDeg>`, `<camDeg>` — absolute arc/camera angles in `[0, 360)`, e.g. `090.000`.
- `LED<NN>` — **0-based** LED index: **L1 = LED00 ... L7 = LED06**.
- `_repMM` — present only when capturing **more than one** photo per LED (New Scan repeats, or any multi-shot run). Single-shot scans have no `_rep`.

- **Manual Save captures** are named `capture_<timestamp>.<ext>`.

Format meanings

Format	What you get
--------	--------------

TIFF	The raw 12-bit sensor data (a Bayer colour mosaic) stored in a 16-bit file, plus a companion .png that is an 8-bit, white-balanced preview. Use the TIFF for measurement; the PNG is for viewing.
PNG / BMP	Processed 8-bit image.
JPEG	Compressed 8-bit image.

The raw TIFF is a colour Bayer mosaic, not a plain greyscale image — adjacent pixels are different colour channels. Any quantitative analysis must treat it per-colour-channel.

JSON sidecars

- **Grid scan** and **New Scan** write **one** JSON per run, `scan_<YYYYMMDD_HHMMSS>.json`, containing:
 - `scan_start_time`
 - `camera_settings` (model, serial, exposure, gain, resolution, pixel format, bit depth)
 - `scan_config` (the full configuration, including resolved LED and camera positions)
 - `images` — one record per captured file, keyed by filename, with target and encoder-measured angles, LED index/angle/brightness, residual errors, repeat index, camera-slit angle, and (for TIFF) the pixel format and bit depth.
- **Manual Save** writes a per-image sidecar next to the image with the current encoder readings, format, and camera-slit angle.

These sidecars are what make the data self-describing — **keep them with the images.**

15. Recommended operating procedure

A reliable session, start to finish:

1. **Start the backend** on the rig PC (`python run.py`) and open the page ([§3](#)).
2. **Connect** to the ESP32 (Serial or TCP) — pill turns green ([§6](#)).
3. **Position to zero and Set Home** for both axes.
4. **Open the camera**, start the **Live preview**, and set **Exposure/Gain** so the sample is well-exposed but **not clipped** (avoid pure-white blowout on bright/specular samples). Apply.
5. Confirm the **camera mounting slit** angle matches the physical mount.
6. Set the **Output Folder** and **Image Format** (TIFF for measurement).
7. Choose your workflow:
 - **Grid scan** — set LED/camera axes, enable LEDs + PWM, check the position and skipped counts, then **Start Scan**.
 - **Single geometry** — use **New Scan Configuration**, set camera angle, relative LED angle, photos per LED, LEDs, then **Start New Scan**.
8. **Watch the first few captures** in the gallery and the Progress card. Use **Pause** if you need to intervene; **Abort** to stop.
9. When done, **collect the image files and their JSON sidecars** from the output folder for analysis.

Exposure sanity check: before a long run, capture one frame of your brightest case and confirm it isn't saturated. A clipped reference frame is unusable for radiometry, and you won't get that geometry back without re-running.

16. Diagnostics: encoder jitter

On the **Encoder Readings** card, **Measure encoder jitter** samples the resting output encoders (camera OME85 and LED-arc AS5600 #3) for a **Window (s)** you choose (default 10 s) and reports each encoder's **peak-to-peak**, **standard deviation**, and **min/max**.

- **Hold the arm perfectly still** during the measurement — do not move the motors.
- Use the result to understand the **noise floor** of the encoders, i.e.

the smallest positioning tolerance that is physically meaningful. If precise positioning "won't converge" to an unrealistically tight target, this tells you why.

17. Troubleshooting

Symptom	Likely cause / fix
Connection pill stays red	Wrong port/IP, cable, or driver. Refresh ports; check nothing else holds the port.
No serial ports listed	ESP32 not detected — reseal USB, check driver.
Camera won't open	IDS Peak SDK missing, camera not plugged into USB 3, or already open elsewhere. Check the log message.
Live preview blank or "unavailable"	Camera not open, or a scan/manual-capture is using it (preview pauses then). Wait or open the camera.
Preview freezes briefly on Apply / Save	Normal — the stream is quieted while camera settings are written or a raw capture retunes the format.
Encoder shows –	That output encoder isn't reporting; moves run open-loop and precise landing can't be verified.
Move "not converged" in log	Target tighter than the encoder noise floor, or mechanical binding. Run the jitter test; loosen expectations or check the axis.
Scan refuses to start (all positions blocked)	Every position has the LED arc blocking the lens. Adjust angles so the camera reaches at/above the LED angles, extend the LED sweep past the block angle, or lower/disable the block angle.

Save Image blocked	A scan is running (it owns the camera). Wait or abort.
Motor turns the wrong way	Tick Reverse direction for that motor.
Code changes not taking effect	Backend changes require restarting python run.py on the rig PC — a browser refresh alone is not enough.
Anything unsafe	Hit E-STOP , then investigate.

18. Safety

- **E-STOP** (top bar) stops all motors and LEDs instantly. Know where it is.
 - **Watch the LED arc vs the camera** when setting up geometry manually — the arc can physically block or collide with the lens. The grid scan's block-angle guard helps, but the **New Scan** panel has no guard.
 - **Don't stare into the LEDs** at high brightness.
 - **Home after any manual repositioning** so the recorded angles are meaningful.
 - Keep hands clear of the arms while a move or scan is running (watch the ● **MOVING** badges).
-

19. Quick reference

Motor map: Motor 1 = Camera · Motor 2 = LED arc **LEDs:** L1...L7 at 0°,10°,20°,30°,40°,50°,60° on the arc · PWM 0–255 · files use LED00...LED06 **Measurement format:** TIFF (raw 12-bit Bayer mosaic + companion PNG preview) **Default output:** `./captures` (on the rig PC) **Home:** position manually → Set Home before measuring **E-STOP:** stops all motion + LEDs immediately **One scan at a time:** grid scan and New

Scan share Pause/Resume/Abort and progress **Occlusion rule (grid scan)**: skip when $0 < (\theta_{\text{LED}} - \theta_{\text{camera}}) \leq \text{block angle}$ (~50°)

--